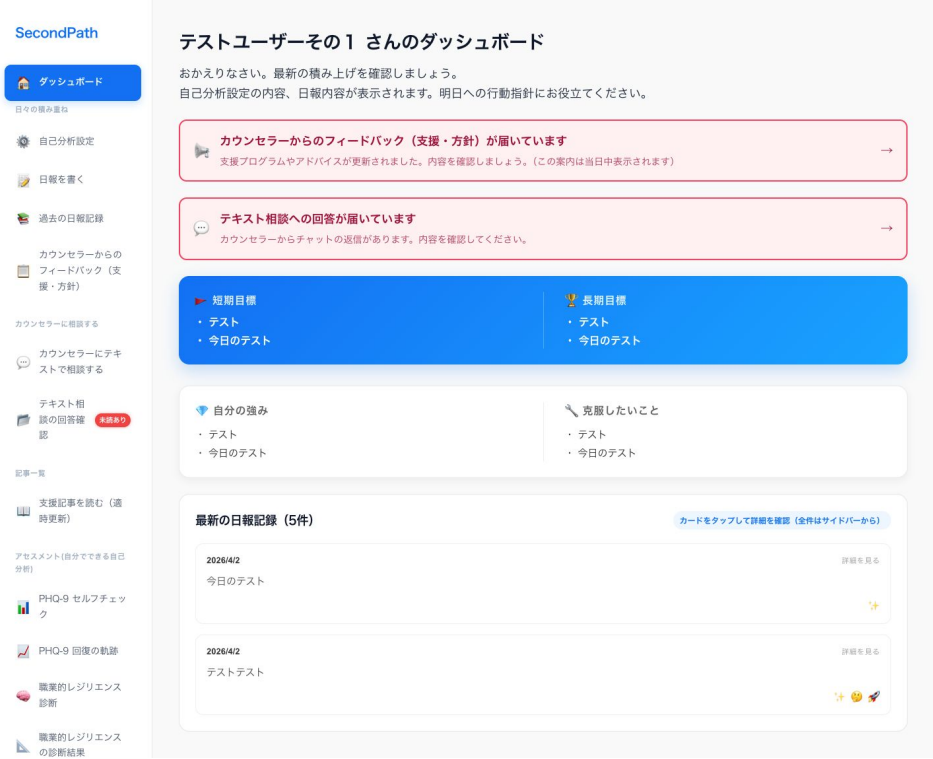
ポートフォリオ
-SecondPath-

Next.js 16 (App Router) × React 19 × Ruby on Rails 8 API



スキルマップ

■ 主要技術

- **Frontend:** Next.js 16 (App Router / RSC / Client分離,feature-based設計), React 19 , TypeScript, CSS Modules
- **Backend:** Ruby on Rails 8 API 連携, 認証フロー整合 (Supabase / Cookie / Header), エラーハンドリング・データ正規化
- **Infra / DevOps:** Docker (Dockerfile / docker-compose), Vercel (Frontend), Supabase (Auth / DB / Storage)
- **Quality:** RSpec,Vitest (単体テスト), Playwright (E2E), ESLint / Prettier / Stylelint

プロジェクト紹介（SecondPath）

■ プロダクトの方針と機能

「カウンセリングの効用、クオリティを高める、高機能業務支援プラットフォーム」

- **マルチテナント設計（利用者・管理者の完全分離）**　ユーザー用のマイページと、管理者用のダッシュボードをそれぞれ独立して実装。役割ごとに最適な画面を提供し、情報の混同を防ぐ確実な管理体制を構築しています。
- **データビジュアライゼーション（診断結果のグラフ化）**　診断やテストの結果をリアルタイムでグラフに反映。数値の羅列ではなく視覚的なフィードバックを行うことで、ユーザー自身の現状把握をスムーズに促します。
- **リアルタイムコミュニケーション（ダイレクトチャット）**　支援者とユーザーが直接やり取りできるチャット機能を搭載。必要な時にすぐ繋がれる環境を用意し、きめ細やかなフォローアップを可能にしています。

技術的ハイライト

採用した設計と技術

- **Feature-based 設計**
  - `src/features/**` に画面・API・型を集約
  - 変更の影響範囲を局所化し、保守性を向上
- **Server / Client 分離（App Router）**
  - 初期データ取得は Server Component で `await`
  - UI操作・状態管理は Client Component に限定
  - 表示速度と責務分離を両立
- **認証状態の一貫性設計**
  - Supabase セッションを単一ソースに統一
  - Cookie / Header（X-User-Id）連携で Server 側も同一のユーザー文脈を維持
- **体感速度を上げる遷移設計**
  - `loading.tsx` + クライアント遷移スピナーで待機時間を可視化
- **品質担保のテスト一部導入**
  - **Vitest（単体）**：日報入力/保存ロジックの検証
  - **Playwright（結合）**：日報投稿フローの自動テスト

テストアカウントなど

Github

FE:https://github.com/tokumei29/SecondPath-frontend

BE:https://github.com/tokumei29/SecondPath-backend

アプリURL

ユーザー側画面 :https://secondpath-app.jp/

テストアカウント　メールアドレス:「test@example.com」 PW:「00000000」

アドミン側画面 :https://secondpath-app.jp/adminDashboard

テストPW:「your_secret_pass」